



Growth Pulse
SMART PLANT MONITORING

Engineering Manual

The development record: architecture, decisions, problems, and fixes

Document	GP-EM-001
Revision	3.0
Date	June 11, 2026
Status	Released
Applies to	Firmware v4.3 / Web app v2.22.2

Contents

About this manual	4
Part I. The Product and the System	5
1. Product overview	5
2. System architecture	5
Part II. Hardware Engineering	10
3. The board	10
4. Pin assignments, and why each one	11
5. Sensor electronics	11
6. Power and battery	13
Part III. Firmware Engineering	15
7. Firmware lineage	15
8. The combined firmware, section by section	15
9. The diagnostic sketches	21
Part IV. Cloud Engineering	22
10. The device cloud (Losant)	22
11. Accounts, ownership, and routing (Supabase)	23
12. The backend (Netlify functions)	25
13. LoRaWAN engineering (The Things Stack)	28
Part V. The Web Application	31
14. Application architecture	31
15. Domain logic	32
16. Views and components	33
17. The report system	34
18. The design system and PWA	34
Part VI. Integration, the Record, and the Road Ahead	36
19. End-to-end sequences	36
20. The debugging journal	37
21. Decision log	40

22. Known limitations and roadmap	41
23. Appendix	42

About this manual

This is the engineering record of GrowthPulse. It documents every layer of the system, the reasoning behind every significant decision, every problem encountered during development, and the workaround or fix that resolved it. A new engineer who reads this manual front to back should understand not just what the system does, but why it is built the way it is, what was tried and rejected, and where the sharp edges are.

It is one of four controlled documents. Each has a distinct job:

Document	Job
Engineering Manual (this document)	The development record: architecture, decisions, reasoning, problems, and fixes
Technical Reference Manual	The current-state specification: pinouts, constants, APIs, schemas, payloads, identifiers
Operations Manual	Procedures and runbooks: setup, flashing, provisioning, deployment, recovery, troubleshooting
User Manual	The customer-facing guide: setup, pairing, app features, troubleshooting, FAQ

When this manual and the Technical Reference Manual disagree on a value, the source code is authoritative and the Technical Reference Manual is the first document to correct.

Credentials policy: no usernames, passwords, keys, or tokens appear in this manual or in git. Live secrets exist in exactly three places: the gitignored `.env` on the development machine, Netlify's environment variable store, and the private firmware build. The repository copy of the firmware carries a placeholder provisioning token.

Part I. The Product and the System

1. Product overview

GrowthPulse is a complete consumer IoT product: a sensor node a customer plugs in and pairs from their phone, a cloud pipeline, and a multi-tenant web application with real accounts, alerts, plant intelligence, and subscription tiers. The live application is at growthpulsecloud.com.

Three hard requirements shaped almost every decision in the system:

1. **One firmware image for every board, with no per-board secrets.** A board figures out its own identity at first boot. This is what makes "mail a customer a board" possible, and it is why self-provisioning exists at both the Wi-Fi and LoRaWAN layers.
2. **The customer never touches a cloud console.** Adding a gateway, switching a node between Wi-Fi and LoRaWAN, factory-resetting a unit for resale: all of it happens inside the GrowthPulse app. Losant, Supabase, and The Things Stack are invisible to the customer.
3. **The app always shows the truth.** A claimed device that has never reported shows "waiting," not invented numbers. A disconnected probe shows "not connected," not garbage. The connection badge shows the link the node is actually using, derived from telemetry, never a label the user picked. Trust is the product.

1.1 What the product does

A node carries three sensors: a DS18B20 waterproof soil temperature probe, a DHT22 air temperature and humidity sensor, and a capacitive soil moisture probe. It streams readings to the cloud over the customer's Wi-Fi, or over LoRaWAN through a gateway for fields and far gardens. The app turns those readings into a live dashboard with a 0 to 100 plant health score, species-aware status colors and advice (a catalog of more than 140 plants, each mapped to ideal ranges), history charts, configurable alarms with email and SMS delivery, weather and rain-delay intelligence pinned to the plant's own home location, a growth journal, and branded PDF reports.

1.2 Development timeline in one paragraph

The app went from first commit to v2.22.2 between June 1 and June 11, 2026; the firmware went from the team's baseline sketch (hardcoded Wi-Fi credentials, serial prints) to the v4.3 combined dual-transport image over the same period. The big arcs were: simulated data to live cloud data (June 1 to 3), real accounts and cloud-authoritative ownership (June 3 to 5), honesty and reporting (June 4 to 5), self-provisioning and battery (June 5 to 9), and the full LoRaWAN system with automated provisioning and bidirectional transport switching (June 9 to 11). Chapter 20 is the complete chronological record of every problem hit along the way.

2. System architecture

2.1 The two clouds, and why there are two

GrowthPulse uses two managed clouds, each doing the thing it is best at.

Losant is the device cloud and the single source of truth the app reads. Every node, Wi-Fi or LoRaWAN, is represented as exactly one Losant device. Losant terminates the MQTT connections from Wi-Fi units, stores the latest state and the full time-series history per device, runs the email and SMS alert workflows, and delivers commands back down to connected units.

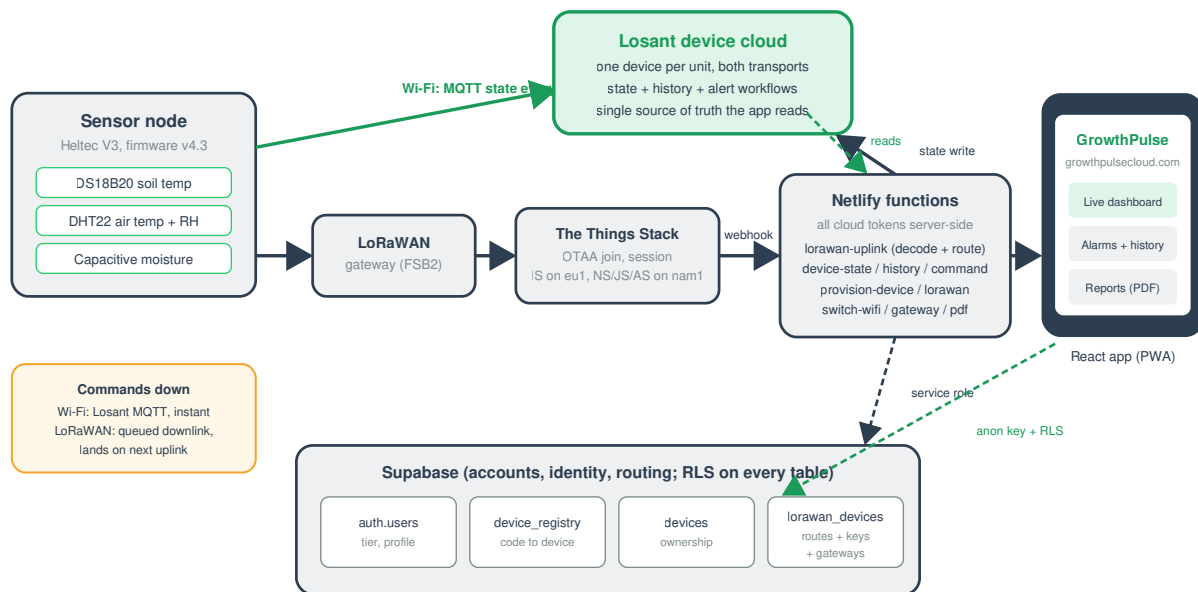
The Things Stack (TTS), on The Things Network sandbox, is the LoRaWAN network server. It owns the radio side: gateways, OTAA joins, and the LoRaWAN session. It stores no plant data. When a LoRaWAN uplink arrives, TTS forwards it to a GrowthPulse webhook, which translates it into a Losant state update. TTS is a bridge from the radio world into Losant, never a second source of truth.

The design choice that makes transport switching possible: **both transports write to the same Losant device**. A node that moves from Wi-Fi to LoRaWAN and back remains one plant in the app the entire time, because the app only ever reads Losant.

```

Wi-Fi node    --MQTT-----> Losant  <--reads-- app
LoRaWAN node --radio--> gateway --> TTS --webhook--> Losant  <--reads-- app
  
```

GrowthPulse System Architecture



Both transports write to the same Losant device, so a node stays one plant in the app no matter how it connects.

2.2 The data path

```
Sensors (DS18B20, DHT22, capacitive soil probe)
-> ESP32-S3 combined firmware (GP_Combined.ino, v4.3)
-> MQTT over Wi-Fi, or a 9-byte LoRaWAN uplink through a gateway
-> Losant device cloud (live state, time-series history, alert workflows)
-> Netlify serverless functions (all cloud tokens held server-side):
    device-state.js    live composite state, polled by the app
    device-history.js  time-series history for report graphs
    render-pdf.js      headless-Chrome vector PDF of a report
-> React web app at growthpulsecloud.com
-> the customer
```

2.3 The control path

```
App action (factory reset, transport switch)
-> Netlify function (owner-verified, command allowlisted)
-> Losant command REST API (Wi-Fi units, delivered over MQTT)
    or a TTS downlink queued for the node's next uplink (LoRaWAN units)
-> firmware command handler / downlink handler
```

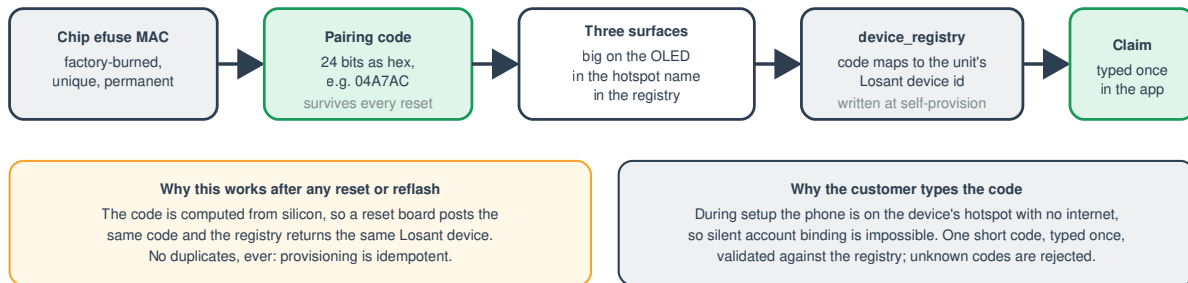
The asymmetry between those two delivery mechanisms, instant on Wi-Fi versus next-check-in on LoRaWAN, drives a large amount of design in chapters 13 and 20.

2.4 The identity chain

```
ESP32 factory-burned MAC (efuse, unique per chip, permanent)
-> 24 bits rendered as uppercase hex = the pairing code
-> shown large on the OLED, embedded in the setup hotspot name
-> device_registry row in Supabase maps pairing code -> Losant device id
-> customer types the code in the app -> claim validated -> device on account
```

The pairing code is deterministic: the same physical board always produces the same code, even after a factory reset or a reflash. That single property is what lets a reset board reclaim its own cloud identity instead of creating a duplicate (section 12.2), and it is why the code doubles as the unit's serial number. The combined firmware derives the code from the high 24 bits of the efuse MAC, zero-padded to six hex characters; the original demo unit, provisioned under the earlier firmware, keeps its historical five-character code `4A7AC` because the registry, not the formula, is the authority once a code is registered.

Identity and Claiming (one permanent identity, three surfaces)



2.5 Services used, and what each one does

Service	Used for
GitHub	Source control; pushing to main triggers production deploys (github.com/BryanPuckettGH/growthpulse-app)
Netlify	Hosting for growthpulsecloud.com, the build pipeline, nine serverless functions, and the environment variable store
Supabase	User accounts (email and password auth), the pairing registry, device ownership, gateway records, LoRaWAN routing
Losant	Device cloud: MQTT broker, state and history storage, email and SMS alert workflows, command delivery
The Things Stack	LoRaWAN network server on The Things Network sandbox: gateways, OTAA, session, uplink webhook
Open-Meteo	Weather forecasts and the place-name geocoder for per-plant home locations; free, no API key

Supporting references: Heltec board documentation (wiki.heltec.org), the Silicon Labs CP210x USB driver (silabs.com), the Arduino IDE (arduino.cc), and the RadioLib LoRaWAN stack (github.com/jgromes/RadioLib).

2.6 The security model in five rules

- No cloud token ever reaches a browser.** The app calls same-origin Netlify functions; the functions hold the tokens. The `VITE_` prefix on an environment variable is the public/private boundary, enforced by Vite at build time.
- The database is the guard.** Supabase row-level security gates every table. The anon key ships in the bundle by design; it grants exactly what the RLS policies say and nothing more. Service-role access, which bypasses RLS, exists only inside serverless functions.
- Least privilege per token.** Losant has a read-only token for the hot polling path and a separate write-capable token for commands and state injection. If the read token leaked, it could only read.

4. **The firmware image carries no per-board secrets.** Wi-Fi identities are minted at first boot by `provision-device`; LoRaWAN keys are pushed at provisioning time and live only in the board's flash. The one shared value, the firmware provisioning token, is a placeholder in the repository and real only in private builds.
 5. **Commands are allowlisted and owner-verified.** The command endpoint accepts exactly the commands it knows, requires a signed-in session, and confirms through RLS that the caller owns the target device before sending anything.
-

Part II. Hardware Engineering

3. The board

Heltec WiFi LoRa 32 V3, silkscreen HTIT-WB32LAF. It was selected because it carries every radio the product roadmap needs on one module:

Component	Detail
MCU	ESP32-S3 dual-core, 240 MHz, 8 MB flash
Radios	2.4 GHz Wi-Fi b/g/n, BLE, and a Semtech SX1262 LoRa transceiver (915 MHz US)
Display	0.96 inch SSD1306 OLED, 128x64, I2C
USB	USB-C through a Silicon Labs CP2102 UART bridge
Buttons	RST (hard reset) and PRG (user button on GPIO0)
Power	5V in via USB-C or the 5V header pin, onboard 3.3V regulator, JST connector and charge circuit for a LiPo battery

The same physical unit is the Wi-Fi node and the LoRaWAN node; the transport is purely a firmware decision. That is the hardware foundation of the whole dual-transport design: no new electronics were needed to add LoRaWAN.

4. Pin assignments, and why each one

Function	GPIO	Reasoning
DS18B20 data (1-Wire)	4	Listed as free and beginner-safe in Heltec's official GPIO guide
DHT22 data	5	Same list
Soil moisture analog out	2	Moved off GPIO1 after bring-up testing: GPIO1 is wired to the board's battery-voltage sense network and read a flat 0 regardless of the sensor. GPIO2 is the adjacent clean ADC1 channel
PRG button	0	The board's built-in user button; a boot-strapping pin, safe to read after boot
Battery sense control	37	Gates the battery voltage divider (section 6.2)
Battery sense ADC	1	The divider's midpoint; the reason the soil probe could not stay on GPIO1
Optional VBUS sense	7	Midpoint of the optional 100k/100k USB-detect divider (section 6.4)
OLED SDA / SCL / reset	17 / 18 / 21	Fixed by board routing
OLED power rail (Vext)	36	The V3 feeds the OLED from a switched rail; firmware must drive GPIO36 LOW to power it. Forgetting this is the classic blank-screen failure

The SX1262 radio pins (NSS 8, DIO1 14, RST 12, BUSY 13, SCK 9, MISO 11, MOSI 10) are fixed by the board schematic and covered with the LoRaWAN bring-up in chapter 13.

Analog constraints worth memorizing: analog inputs must use ADC1 channels (GPIO1 through 10) because ADC2 is owned by the Wi-Fi radio while it runs. GPIO19 and GPIO20 are therefore unusable for the soil probe. Resolution is configured to 12 bits, so raw readings span 0 to 4095.

5. Sensor electronics

5.1 DS18B20, soil temperature

A digital 1-Wire device in a waterproof probe. Wiring: red to 3V3, black to GND, yellow (data) to GPIO4, plus a **4.7 kilo-ohm resistor from the data line to 3V3**. The resistor is not optional, and the reason is the bus design: on 1-Wire, the master and every sensor only ever pull the line LOW. Nothing on the bus can drive it high. The pull-up resistor is the only thing that returns the line to 3.3V between pulses. Without it the line never rises, communication is impossible, and the Dallas library returns its "no device" sentinel of -127 C, which converts to the famous **-196.6 F**. That exact value was the first hardware mystery of the project; the app now recognizes it (section 16.2) and turns it into a "probe not detected" insight that names the resistor.

Each DS18B20 carries a unique 64-bit serial number whose first byte (the family code) is 0x28. During bring-up we used a 1-Wire scanner sketch that searches the bus and prints these

addresses; a found address proves the wiring end to end even if temperature conversion were broken, and a storm of random non-0x28 addresses indicates electrical noise on a floating line rather than real devices.

5.2 DHT22, air temperature and humidity

A single-wire (not 1-Wire) digital sensor: VCC to 3V3, data to GPIO5, 10 kilo-ohm pull-up from data to VCC. The common 3-pin breakout module has the resistor built in; a bare 4-pin part needs it added. When the sensor is absent the library returns NaN; ArduinoJson serializes NaN as JSON null; the connector passes null through; and the app reads null as "not connected." The failure mode was designed into the pipeline rather than hidden.

5.3 Capacitive soil moisture probe

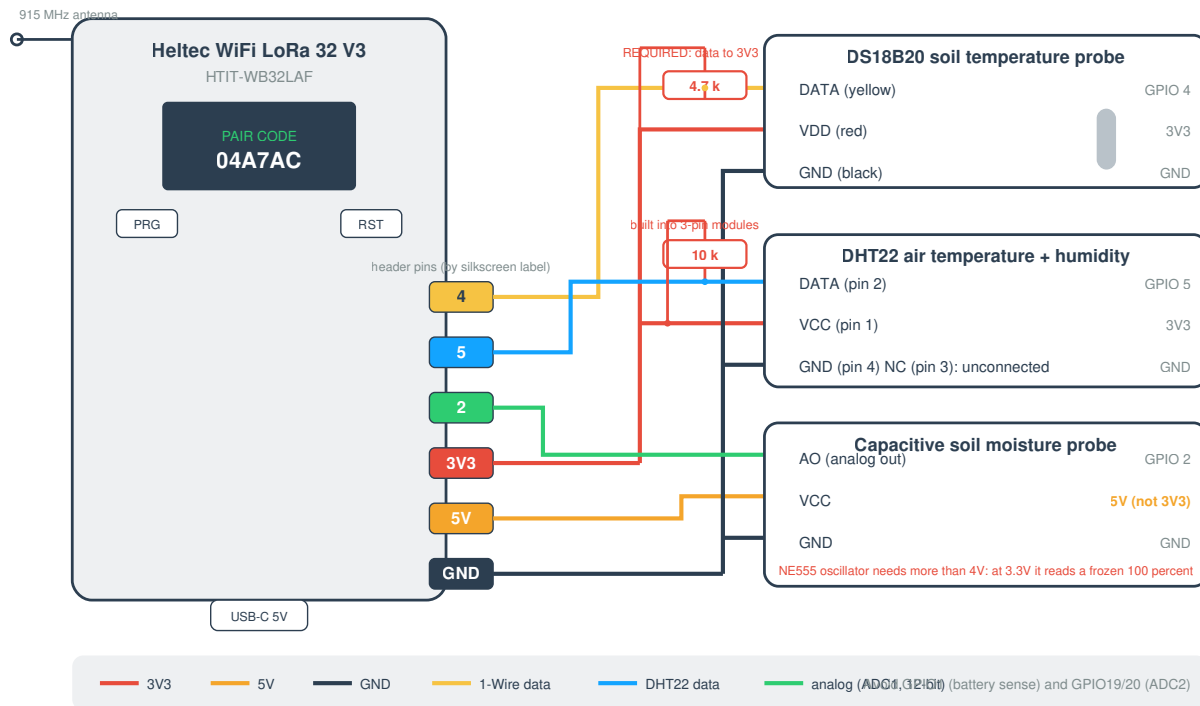
The flat-blade capacitive probe (v1.2 style): VCC, GND, AO (analog out). Two hard-won findings:

1. **It must be powered from 5V, not 3.3V.** These modules build their oscillator around an NE555 timer, and the NE555 does not run below roughly 4 volts. At 3.3V the power LED lights (LEDs do not care) but the oscillator is dead and AO sits near 0.1V, which reads as raw ~120 and maps to a frozen "100 percent" moisture. Vendors list the modules as 3.3 to 5.5V; the ones with NE555 silicon are 5V parts in practice. At 5V supply the analog output tops out near 3V, inside the ESP32 ADC's safe range, so no level shifting or divider is required.
2. **Raw counts, not percent, are the calibration currency.** The firmware reports both. Calibration: read the raw value with the probe in dry air (that becomes `dryValue`, typically near 3600) and submerged to its line in water (`wetValue`, typically near 1300), then update the two constants. Note the inversion: wetter soil means lower raw counts, inherent to how the capacitance loads the oscillator.

The firmware additionally treats a raw reading below 300, or a 10-sample spread wider than 600 counts, as "no probe": a powered probe in air sits near its dry point, so a near-zero or wildly jittering signal is a floating pin, not data (section 8.6).

5.4 Wiring summary

GrowthPulse Node Wiring (Heltec WiFi LoRa 32 V3)



Sensor	VCC	GND	Signal	Extra
DS18B20	3V3	GND	GPIO4	4.7k pull-up, data to 3V3, required
DHT22	3V3	GND	GPIO5	10k pull-up (built into 3-pin modules)
Soil moisture	5V	GND	GPIO2	none; powered from 5V on purpose

The bench-level wiring procedure, including bench supply settings and the pin-finding guide for the board's silkscreen, lives in the Operations Manual.

6. Power and battery

6.1 Supply

The board takes 5V in: USB-C from any phone charger, or a bench supply at 5.0V with roughly a 1A current limit feeding the 5V and GND header pins. Measured behavior: about 150 mA idle, with 300 to 500 mA bursts during Wi-Fi transmit. Brownout resets during flashing or Wi-Fi joins are a cable or supply problem, not a firmware problem. Sensors draw single-digit milliamps from the board's regulated rails. Never feed 5V into a 3V3 pin.

6.2 Battery sensing, and the inverted control pin

The Heltec V3 reads its LiPo through a voltage divider gated by a control pin (GPIO37), with the midpoint on GPIO1. Two findings from bring-up:

- **On this board the control polarity is inverted from the documented Heltec reference.** Driving the pin per the documentation disabled the divider exactly when the firmware meant to enable it, which is why the battery always read 0 percent and the unit claimed AC power even on battery. The firmware drives the opposite level, waits for the divider to settle, averages 16 ADC samples, then releases the pin to an input.
- The measured conversion is `volts = rawAverage / 238.7`, mapped to percent through a 100-entry discharge curve between 3.04V (empty) and 4.26V (full). A simple linear voltage-to-percent map reads LiPo packs badly because their discharge curve is flat in the middle; the lookup table tracks the real chemistry.

6.3 Charging inference and display smoothing

The V3 exposes no charge-status pin, so charging cannot be read; it is inferred. The firmware smooths the measured voltage (an exponential filter), then reasons: a rising trend means charging; a voltage sitting at the charger's top-of-charge level means charging (or done); a falling trend means discharging. Two refinements came from observed misbehavior:

- **A charger inflates the pack's terminal voltage instantly**, so a 44 percent pack read as 65 percent the moment a cable went in. The displayed percent now creeps at most 1 percent per reading, rising only while charging and easing back toward the true level on battery. What the customer sees tracks the real charge, not the electrical artifact.
- **With no battery installed on USB**, the charge chip pins the sense line near a full pack's voltage, and there is no battery-detect pin to tell "no battery" from "full battery." This cannot be fully resolved in software. The firmware reports AC power when the line sits pinned at the top and is no longer rising, which is honest for both the no-battery case and the fully-charged-on-charger case.

6.4 The optional VBUS mod, true USB detection

The fully honest fix for USB-versus-battery detection is one optional hardware mod: a 100k/100k divider from the board's labeled 5V pin to ground, midpoint on GPIO7. On USB the midpoint reads about 2.5V; on battery, 0V. With the firmware's `USE_VBUS_SENSE` flag set to 1, the unit reads a real plugged-in signal instead of inferring from voltage. Default is off, so boards without the wire are unchanged. The wire is tapped from the 5V header pin, never from inside the USB-C cable.

6.5 Soft power-off

The board has no power switch. A double-tap of PRG drops the unit into ESP32 deep sleep: the OLED rail is powered down, wake is armed on the PRG pin, and draw falls to microamps, so a battery lasts months. A single press (or RST) wakes it; the firmware detects the deep-sleep wake cause and skips the boot-time Wi-Fi-wipe check so waking never destroys credentials (an early bug, fixed in firmware v3.9).

Part III. Firmware Engineering

7. Firmware lineage

The firmware evolved through four named generations. Each exists in the repository under `firmware/` because each remains the cleanest reference for the layer it introduced.

Generation	Versions	What it introduced
Team baseline	v0.9	The original team sketch: sensor reads with hardcoded Wi-Fi credentials and serial prints
GP_Provisioning	v1.0 to v2.3	Branded captive-portal Wi-Fi setup, chip-derived pairing code, OLED UI, Losant streaming, remote factory reset, watchdog, boot self-test, rotating status pages, <code>wifiRssi</code> telemetry
GP_Node	v3.0 to v3.11	Self-provisioning (no per-board secrets), then the entire battery system: corrected sense polarity, charging inference, percent smoothing, deep sleep, no-battery honesty, the optional VBUS mod
GP_Combined	v4.0 to v4.3	One image, both transports: the SX1262 LoRaWAN stack, NVS mode selection, OTAA with persisted nonces, the 9-byte payload, downlink handling, <code>provisionLoRa</code> key push, the tiered PRG button, stale-command grace, the authoritative transport tag, on-screen firmware version

`GP_Combined/GP_Combined.ino` (v4.3) is the production image and the only one flashed onto shipping units. The walkthrough below covers it; where a mechanism was inherited unchanged from an earlier generation, the original reasoning is given here once.

Two copies of the combined image exist on purpose: the working copy at the project root carries the real provisioning token and is never committed; the repository copy under `firmware/` is identical except for a placeholder token. Every firmware change is made to both, and the build is verified balanced (compiling, matching versions) before commit.

8. The combined firmware, section by section

The image is a single annotated file of about 900 lines. One file is a deliberate choice at this scale: it can be reviewed top to bottom, pasted whole into the IDE, and handed to a teammate without explaining a build system. It merges two complete network stacks, so it overflows the default app partition; it is built with Tools, Partition Scheme set to a larger-app ("Huge App") layout.

8.1 Boot path selection

A `netmode` string in the NVS namespace `gp` selects the boot path: `wifi` (the default) or `lorawan`. Setting the mode always means writing NVS and rebooting; the firmware never tries to hot-swap

stacks, because RAM is the real ceiling and only the chosen stack is ever initialized. The mode is set three ways, each writing the same flag:

- the setup portal's Connection field (type `lorawan` during Wi-Fi setup),
- a Losant `setMode` command (reaches units that are on Wi-Fi),
- a LoRaWAN downlink (reaches units that are on LoRaWAN).

A LoRaWAN boot with no stored keys falls back to Wi-Fi setup rather than bricking: a unit that cannot join anything always ends up somewhere a human can reach it.

8.2 Identity helpers

`chipUid()` extracts 24 bits of the factory-burned efuse MAC; `pairCode()` renders them as six uppercase hex characters, and `apSsid()` builds the setup hotspot name `GrowthPulse-XXXXXX` from the same value. One identity, three surfaces: the screen, the hotspot name, and the registry. The MAC is unique per chip, free, and permanent, which is the entire provisioning cost model: identity costs nothing at manufacture.

8.3 The OLED system

The V3's OLED hangs off a switched power rail: `oledInit()` drives Vext (GPIO36) LOW, waits 80 ms for the rail to settle, then initializes U8g2 with the V3's exact reset/clock/data pins. The generic constructor assumes default I2C pins and produces a blank screen, a classic first-day failure.

In steady state the screen cycles through three pages, five seconds each:

Page	Content
Pair code	"PAIR CODE", the link state chip ("Wi-Fi ON", "LoRaWAN .."), the firmware version in small type under the link chip, and the code itself rendered with adaptive type
Connection	The live link and its real signal: Wi-Fi dBm, or LoRa RSSI once joined, plus battery percent with charging marker, or "AC/USB"
Readings	Air temperature and humidity, soil temperature, moisture percent; any absent probe renders "--" honestly

`drawBigCentered()` implements the adaptive type: it measures the code in bold 30, then 25, then 20 point, and draws the largest that fits 124 pixels. The pairing code is the one thing a customer must read off the hardware, so it gets the biggest legible rendering. The code never disappears after pairing, on purpose: re-pairing, resale, and support all need it, and it doubles as the serial number.

The firmware version on the pairing page (added in v4.3) earns its place: it confirms at a glance which build a unit runs, and after a reflash the new number on screen is the proof the flash took, with no serial monitor needed.

Burn-in protection dims the panel after five idle minutes; any PRG tap restores full contrast. A boot self-test screen (held ten seconds so it can actually be read) shows OK or "--" per sensor before the unit goes online, which catches miswired benches before anyone stares at cloud dashboards.

8.4 The branded captive portal

Two raw-string constants are injected into WiFiManager's portal: a `<style>` block restyling the stock pages with brand colors, and a branding block carrying the GrowthPulse logo as inline SVG, the wordmark, the tagline, and a welcome line. Inline SVG because the customer's phone is connected to the device's hotspot with no internet: every byte the portal shows must be served from the chip itself.

`connectWiFi()` encodes the provisioning policy in one WiFiManager call plus carefully chosen settings:

- `autoConnect()` tries saved credentials, opens the branded hotspot if none work, and returns once associated. No custom state machine.
- `setWiFiAutoReconnect(false)` and `WiFi.setSleep(false)` fix a real observed failure: while the portal is open the chip runs AP and STA simultaneously, and both background STA retry scans and modem power-save pull the radio off the AP channel. The symptom was the customer's phone hanging on "connecting..." to the hotspot for a minute or more. With these two lines the join completes in seconds.
- `setConfigPortalTimeout(600)`: the 180-second default expired mid-demo while a phone was slow to surface the portal, restarting the AP underneath the user. Ten minutes is a human-paced window; on timeout the device restarts and reopens setup rather than wedging.
- The portal also answers at **192.168.4.1** directly, the documented fallback when a phone suppresses captive-portal detection.
- A "Connection" text field on the portal lets a user (or installer) type `lorawan` to flip the unit's mode at setup time.
- The task watchdog is detached around `autoConnect()` (which legitimately blocks for minutes) and re-attached after.

8.5 Wi-Fi self-provisioning on the device

On the first Wi-Fi boot there is no saved Losant identity in NVS. `ensureProvisioned()` then calls `provisionFromBackend()`: an HTTPS POST of `{ code, token }` to the `provision-device` function, where `code` is the pairing code and `token` is the shared firmware provisioning token. The response carries a Losant device id, access key, and access secret, which are written to NVS and used for every subsequent boot. Provisioning retries every five seconds until it succeeds, with an OLED message pointing at the likely cause ("check internet").

The TLS connection currently uses `setInsecure()` (no certificate pinning) to keep bring-up simple; pinning is a documented pre-production hardening item, because an active man-in-the-middle could otherwise intercept the provisioning response.

8.6 Sensor reading

`readSensors()` is shared by both transports:

- DS18B20 and DHT22 failure sentinels (-196.6 F, NaN) pass through untouched; the cloud and app layers convert them to honest UI rather than the firmware guessing.

- The soil ADC is averaged over 10 samples 10 ms apart, because single ESP32 ADC reads jitter by several counts. The 10-sample min/max spread doubles as a noise detector: a spread above 600 counts, or an average below 300, means no probe is connected (a floating pin reads low noise; a powered probe in air reads near its 3600 dry point), and the reading is zeroed so the pipeline reports "not connected" instead of noise.
- `map(soilRaw, dryValue, wetValue, 0, 100)` runs dry-to-wet, which inverts the scale correctly since wet soil reads lower, and `constrain` clamps readings outside the calibration window instead of reporting 112 percent.
- The battery percent is refreshed in the same pass (chapter 6).

8.7 Wi-Fi telemetry

`sendTelemetryWiFi()` publishes a JSON state document over MQTT: the five sensor values, `wifiRssi`, `batteryPct`, `charging`, and `transport: "wifi"`. The transport tag is stamped on every report and is authoritative: it is what stops a stale `lorarssi` left in the cloud composite from ever making the app display the wrong link after a switch (chapter 20, issue L8 family). The serial line mirrors one summary per cycle, which is the field calibration tool: no extra software needed to calibrate a probe.

The Wi-Fi loop sends every 3 seconds. That cadence is human-real-time for demos and stays far inside Losant limits; LoRaWAN has fundamentally different rules (section 8.9).

8.8 Cloud commands and the stale-command grace window

`handleCommand()` accepts exactly three commands and ignores everything else:

Command	Payload	Effect
<code>factoryReset</code>	none	Wipe saved Wi-Fi, reset mode to Wi-Fi, reboot into the setup hotspot
<code>setMode</code>	<code>{ "mode": "wifi" or "lorawan" }</code>	Write the mode flag and reboot into that stack
<code>provisionLoRa</code>	<code>{ joinEUI, devEUI, appKey }</code> as hex strings	Validate lengths, store the OTAA keys in NVS, reboot into LoRaWAN

All three are gated by a 12-second grace window after connecting to Losant. The reason is chapter 20, issue L9: the broker redelivers a queued command the instant a device reconnects, so a stale `provisionLoRa` or `factoryReset` from an earlier session would yank a unit into a mode change, or wipe its Wi-Fi, on every connect, producing an endless Wi-Fi/LoRaWAN bounce. A real owner action arrives during steady operation; anything delivered in the first seconds of a connection is a replay and is logged and dropped.

`factoryReset` is the device half of the resale flow: the owner taps Factory reset in the app, the cloud delivers the command over the existing MQTT connection, and the unit visibly wipes itself and reopens setup for the next owner. Commands reach online devices only; there is no offline queue on the Losant path, which is why the app's confirmation dialog documents the PRG-button fallback.

8.9 The LoRaWAN path

Radio bring-up. The SX1262 needs three V3-specific facts to even start: the exact SPI pins, a non-zero TCXO voltage (the V3 clocks the radio from a temperature-compensated oscillator powered through the radio's DIO3 pin; RadioLib's 1.6V default works, and a zero value produces init error -707), and `setDio2AsRfSwitch(true)` because DIO2 drives the antenna switch (without it, transmit appears to work and nothing is ever received). `setDutyCycle(false)` is correct for US915, which uses fair-use airtime rather than a legal duty cycle.

OTAA, the two-call trap. In RadioLib, `beginOTAA()` only sets the keys and returns success without joining; `activateOTAA()` performs the actual join. The combined image originally called only the first, so keys were set, no join ever happened, and the first uplink failed with -1101 "session not active" (chapter 20, issue L2). The fix matches the proven standalone sketch: restore saved join nonces from NVS, `beginOTAA`, then loop on `activateOTAA` with a 10-second backoff until joined, persisting nonces between attempts so the DevNonce counter always increments. The PRG button stays responsive inside the retry loop, because a node that cannot join must still be recoverable by hand.

DevNonce persistence. LoRaWAN 1.0.4 requires each join request to carry a DevNonce higher than any the network has seen. The nonces live in a dedicated NVS namespace and are saved after every join attempt and every uplink. A reflash that wipes NVS restarts the counter, which the network would silently reject until the counter caught up; the backend closes that hole by setting `resets_join_nonces` on the Join Server at provision time (chapter 20, issues L13 and L14).

The 9-byte uplink. LoRaWAN frames are tiny (11 bytes of application payload at the slowest US915 data rate), so the node packs binary, not JSON:

```
[0..1] soilTemperatureF * 10, int16 big-endian    (0x8000 = sensor absent)
[2..3] airTemperatureF * 10, int16 big-endian
[4]   airHumidity percent,      uint8
[5..6] soilRaw 0..4095,         uint16 big-endian
[7]   soilMoisturePercent,      uint8
[8]   batteryPct 0..100,        uint8            (0xFF = unknown)
```

LoRaWAN Uplink Payload (9 bytes, fPort 2, big-endian)



Absent-sensor sentinels

soil temperature: 0x8000 means no probe (the webhook converts it to null)
 battery: 0xFF means unknown / AC powered
 soilRaw 0 means no probe detected (raw under 300 or noisy spread)

Why 9 bytes

US915 DR0 (slowest, longest range) allows 11 application bytes, so this frame transmits at every data rate. Decoded server-side by lorawan-uplink, with no per-device TTS formatter required.

Downlink: a single byte 0x00 in the same exchange means "switch back to Wi-Fi"; the node writes its mode flag and reboots.

Nine bytes fits every data rate including DR0, so the frame transmits even at maximum range. Uplinks go out on fPort 2 using the 8-argument `sendReceive()` form, which captures any downlink

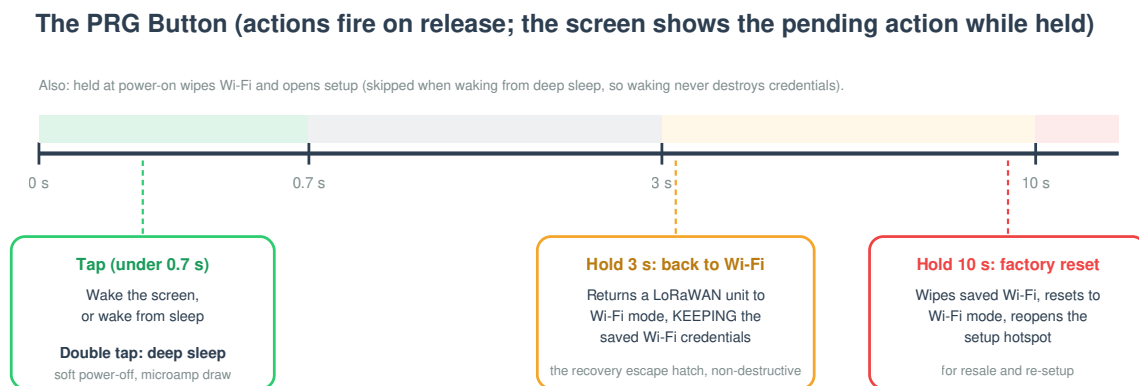
in the same call along with its receive window; RSSI and SNR are read from the radio after each exchange and shown on the OLED and reported upstream.

Downlinks. A one-byte `0x00` downlink means "switch back to Wi-Fi": the firmware writes the mode flag and reboots. It acts immediately and unconditionally, because the backend guarantees the queue is clean: `provision-lorawan` clears any stale queued downlink when it provisions, so any `0x00` that arrives is a real, deliberate switch. An earlier firmware-side guard (ignore a `0x00` in the first 20 seconds after a join) was removed in favor of that backend fix, because the guard had also been suppressing legitimate switches (chapter 20, issue L12).

The uplink interval. `LORA_UPLINK_MS` is 60 seconds in the current build, and this number is a deliberate, documented compromise. A class-A node can only receive a downlink in the brief windows after it uplinks, so the uplink interval bounds the latency of every app-to-node action, including switch-to-Wi-Fi. Sixty seconds makes the bench and demos responsive. The Things Network fair-use policy (30 seconds of airtime per device per day) calls for roughly 15-minute intervals in production, accepting the corresponding switch latency. The constant carries that instruction in a comment, and it is the one open production-hardening item (chapter 22).

8.10 The PRG button state machine

One button, four behaviors, acting on release, with a live OLED hint while held so the owner always knows which action they are about to get:



Gesture	Action	Why it exists
Single tap	Wake the screen (or wake from deep sleep)	Burn-in dimming needs a wake gesture
Double tap	Enter deep sleep, the soft power-off	The board has no power switch
Hold 3 seconds	Graceful return to Wi-Fi mode, keeping saved Wi-Fi credentials	The escape hatch from a stuck LoRaWAN unit (section 13.6); non-destructive on purpose
Hold 10 seconds	Full factory reset: wipe Wi-Fi, reset mode, reopen setup	Resale and re-setup

The 3-second hold originally performed the destructive Wi-Fi wipe, which made the only LoRaWAN escape hatch cost the customer their Wi-Fi setup. Splitting the tiers (v4.1) broke the recovery deadlock without sacrificing the reset (chapter 20, issue L10). The handler is polled (no interrupts), which is trivially safe alongside two radio stacks, and every blocking wait in the firmware loops over `handleButton()` so a hold is never missed.

A boot-time check still wipes Wi-Fi if PRG is held during power-on, with one refinement: waking from deep sleep skips the check, because the wake button and the reset button are the same physical button and a wake must never destroy credentials.

8.11 Watchdog and main loops

A 60-second task watchdog reboots the unit on any firmware hang; it is fed in every loop and wait, and detached only around WiFiManager's legitimately-blocking portal. The Wi-Fi loop is self-healing first (reconnect whichever layer dropped), then read, send, service MQTT, update the display, and a button-aware 3-second wait. The LoRaWAN loop is read, uplink (capturing any downlink), then a button-aware wait of the uplink interval. Both keep the page rotation and dimming alive during waits.

9. The diagnostic sketches

Two single-purpose sketches live in `firmware/diagnostics/` and are flashed temporarily over the main image for bench work:

- **ADC_Scanner** prints raw values for GPIO 1, 2, 3, 6, 7 side by side twice a second. Used with a jumper test (touch 3V3 to a pin, expect ~4095; touch GND, expect ~0) to prove an ADC pin works and to find which pin a wire actually lands on. This is the sketch that isolated the GPIO1 battery-sense conflict.
- **OneWire_Scanner** searches pins 4, 6, 7 for 1-Wire devices and prints their 64-bit addresses. A real DS18B20 announces with family code 0x28; silence on all pins means the probe never reaches the bus (power, contact, or pull-up); a burst of random addresses is line noise being misparsed, which actually proves the pin and code are alive.

Re-flashing the main firmware restores normal operation. Wi-Fi credentials and the Losant identity survive re-flashes in NVS unless explicitly reset; LoRaWAN nonces survive too, which matters for join behavior.

Part IV. Cloud Engineering

10. The device cloud (Losant)

10.1 Devices and attributes

One Losant application contains one device per physical unit, created automatically by `provision-device`. The attribute schema mirrors telemetry exactly:

Attribute	Type	Source
airTemperatureF, airHumidity	number	DHT22
soilTemperatureF	number	DS18B20 (carries the -196.6 sentinel when unplugged on Wi-Fi; nulled by the webhook on LoRaWAN)
soilRaw	number	raw soil ADC, kept for calibration and diagnostics
soilMoisturePercent	number	derived percentage
batteryPct, charging	number, boolean	the battery system
wifiRssi	number	Wi-Fi link signal
loraRssi, loraSnr	number	LoRaWAN link signal, written by the uplink webhook
transport	string	"wifi" or "lorawan", stamped authoritatively by whichever path reports

A Losant rule that shaped the system: **a state report that includes an attribute the device does not define is rejected in its entirety**. A device created before `loraRssi`, `loraSnr`, and `transport` existed therefore silently dropped every LoRaWAN report while accepting Wi-Fi ones, which was the deepest bug of the LoRaWAN bring-up (chapter 20, issue L8). The provisioning function now re-applies the full attribute schema on every call, not just at creation, so existing devices self-heal on their next check-in.

10.2 Credentials model: keys versus tokens

Two different credential systems, never interchangeable:

- **Access keys** (key + secret pairs) authenticate **devices** over MQTT. Each board's key is minted by `provision-device`, scoped by allowlist to that one device, and lives only in the board's NVS.
- **API tokens** authenticate **applications** over REST, and three exist on purpose: a read-only token for the hot `device-state` / `device-history` polling path (least privilege where call volume is highest), a write-capable token for commands and webhook state injection, and a provisioning token with `device-create` and `key-create` rights used only by `provision-device`.

10.3 The wire protocol

Telemetry publishes to `losant/{deviceId}/state`; commands arrive on `losant/{deviceId}/command` and surface through the client's `onCommand` callback. Command delivery requires the device to be connected at that moment; there is no offline queue. The backend reads state through REST (`GET .../devices/{deviceId}/compositeState`), which returns per-attribute `{ value, time }` pairs, and the webhook writes LoRaWAN state through REST (`POST .../devices/{deviceId}/state`).

10.4 Alert workflows

Email and SMS alerts are Losant visual workflows: a Device State trigger, a conditional holding the rule (for example `{{ data.soilMoisturePercent }} < 25`), and an email or SMS node on the true branch. Two operational facts learned the hard way: the conditional's true output must wire forward into the notification node (wiring the email node backwards into the conditional silently does nothing), and Losant email sends are rate-limited to one per minute, so two qualifying readings seconds apart produce one email, which looks like a failure and is not. These cloud workflows fire even when nobody has the app open; the in-app alarm system (chapter 16) is a separate, complementary layer.

10.5 The simulator

`growthpulse-simulator/simulator.mjs` connects with the same MQTT credential model and publishes realistic randomized telemetry. It built the entire cloud pipeline before hardware existed and remains the demo tool when the physical unit is elsewhere. Operational rule: never run it against the same device id as live hardware; the interleaved sources produce nonsense.

11. Accounts, ownership, and routing (Supabase)

Supabase provides authentication and four small tables. None store sensor data; they store identity, ownership, and routing.

11.1 Authentication

Email/password auth via `@supabase/supabase-js`. Signup attaches profile metadata (first and last name, grower type) that flows into the UI; sessions persist in browser storage and are observed with `onAuthStateChange`, so a refresh never logs anyone out. Password reset is a branded email link flow. The subscription tier also lives in user metadata, which is what made plans finally persist across logins (chapter 20, issue A3).

11.2 device_registry: the factory database

```
create table device_registry (  
  claim_code      text primary key,  
  losant_device_id text not null,  
  created_at      timestampz default now()  
);
```

Every provisioned unit has one row mapping the code on its screen to its cloud identity. RLS is the access model: authenticated users may SELECT (that is what claiming is), and nobody can write through the public anon key because no insert or update policy exists. Writes happen only through the service role, inside `provision-device`. Because the pairing code is chip-derived and permanent, the registry is also what makes provisioning idempotent: a factory-reset board posts the same code and gets its same device back.

11.3 devices: cloud-authoritative ownership

The registry says which codes are valid; the `devices` table says who owns what. One row per claimed unit, owned by `user_id`, with per-user RLS on all four verbs, carrying everything that should follow the account: name, location, geocoded `geo`, group, transport, plant, the full irrigation config, and the optional photo.

Design points that matter:

- `claim_code` is **UNIQUE**, so a second account claiming the same code hits a 23505 violation, surfaced as "already claimed by another account." Claim exclusivity in one constraint.
- The browser talks to this table directly with the anon key; RLS means it can only ever see or change its own rows. There is no trusted server in the path; the database is the guard. It is also what `device-command` authorizes against: the function forwards the caller's JWT to this table, and zero returned rows means 403.
- `grp` is the column name because `group` is a SQL reserved word; the store maps it.
- `created_at` is read into the app as `claimedAt`, which drives the report's "Everything" range and the activity timeline.
- Removing a device deletes the row, which releases the claim for re-claiming (resale).

11.4 gateways and lorawan_devices

`gateways` stores each account's LoRaWAN gateways (name, EUI) with per-user RLS, so gateways follow the account like devices do.

`lorawan_devices` (the v2 schema) is the LoRaWAN routing table, and its shape encodes the most important architectural correction of the project:

```
tts_device_id    text primary key
dev_eui         text not null
app_key         text           -- OTAA root key, re-pushed on reuse
losant_device_id text not null  -- UNIQUE: one TTS device per board
user_id         uuid
```

The UNIQUE index on `losant_device_id` enforces **one TTS device per board**, and the stored `app_key` is what allows provisioning to re-push the same keys instead of minting new ones (chapter 20, issue L6). RLS is enabled with no client policies: the browser can never read or write LoRaWAN routes or keys; only the serverless functions, using the service role, touch this table.

12. The backend (Netlify functions)

Netlify hosts the static app and nine serverless functions, bundled with esbuild per `netlify.toml`. The functions exist for one structural reason: no Losant, TTS, or Supabase service token may ever reach a browser. Three groups:

12.1 The read and write paths

device-state.js is the hot path the app polls. It reads the Losant composite state with the read-only token, converts anything missing or mistyped to null (the app's universal "no data" signal), and returns the reading with `Cache-Control: no-store`, because a stale reading is worse than an extra request. The timestamp rides on the moisture attribute's report time so the app can show data age honestly. It forwards every attribute in the schema, including transport and both signal pairs, which is what lets the app display the true link.

device-command.js is the guarded write path, with four defenses: POST only; a command-name allowlist; the dedicated write token; and an ownership check, in which the function forwards the caller's Supabase JWT to the `devices` table and lets RLS decide (no auth header is 401, zero rows is 403). Only then does it call the Losant command API.

device-history.js backs the report graphs. The app holds only a 60-reading in-memory ring, so anything longer comes from Losant's time-series store. It auto-scales resolution so a report never carries more than about 240 points regardless of span, cleans the same disconnected-sensor sentinels handled everywhere else (so a dead probe leaves an honest gap rather than dragging averages), trims empty lead and tail buckets, and keeps interior gaps so offline stretches stay visible.

render-pdf.js renders a finished report's HTML to a true vector PDF with headless Chrome (a Lambda-sized Chromium plus puppeteer-core, shipped outside the esbuild bundle via `netlify.toml`). It is auth-gated, requiring a valid Supabase session, so it cannot be abused as an open HTML-to-PDF service. It is the preferred download path; the client falls back to in-browser rendering whenever it fails, so the download always succeeds.

12.2 The provisioning pair

provision-device.js is Wi-Fi self-provisioning, the function that makes one-image manufacturing work:

1. Authenticate the board by the shared firmware token.
2. Normalize the pairing code and look it up in `device_registry` with the service role.
3. Unknown code: create a Losant device named `GrowthPulse <CODE>` with the **full** attribute schema, and register the code-to-device mapping. Known code: reuse the existing device, so a re-provision after a factory reset or NVS wipe never creates a duplicate.
4. **Re-apply the full attribute schema on every call** (a PATCH, best-effort). This is the self-heal that fixed the silently-dropped LoRaWAN state (chapter 20, issue L8): a device created before newer attributes existed gains them on its next provision.
5. Mint a fresh access key allowlisted to exactly this one device, and return the triplet.

The shared token's blast radius is deliberately small (it can only create a device and a single-device key), and per-batch tokens, rate limiting, and certificate pinning are the documented production hardening steps.

provision-lorawan.js is the LoRaWAN twin, called when the app switches a node to LoRaWAN, and its defining property is **idempotency: one TTS device per board**.

1. Validate the signed-in user, then look the board up in `lorawan_devices` by its Losant id.
2. **Route exists (with a stored AppKey): reuse it.** Clear the TTS downlink queue (so no stale switch-to-Wi-Fi `0x00` can bounce the node the moment it joins), set `resets_join_nonces` on the **Join Server** (so a reflashed board joins on the first try), and re-push the same stored keys to the board through a Losant `provisionLoRa` command. No new device, no orphan, a stable downlink target.
3. **No route: mint an identity.** Generate a random DevEUI and AppKey (JoinEUI zeros), create the TTS device through the four-call API (section 13.3), store the route with the AppKey, and push the keys to the board.

Two failure-handling details are load-bearing: a non-2xx Supabase response on the route insert is checked explicitly and surfaced (a silently-lost route leaves the webhook unable to map the board, chapter 20 issue L5), and the AppKey is never echoed back to the browser.

12.3 The LoRaWAN bridge functions

lorawan-uplink.js is the webhook TTS calls on every uplink, and it is deliberately self-sufficient:

1. Verify the shared `X-Webhook-Token` header.
2. Take the TTS-decoded payload if a formatter set one, otherwise **decode the 9 raw bytes itself** from `frm_payload`. This removes any dependency on per-device TTS payload formatters, which auto-provisioned devices do not have.
3. Resolve the target Losant device from `lorawan_devices` by the uplink's TTS device id (with a static env-map and a default as fallbacks for the bench era).
4. Build the same state shape the Wi-Fi path sends, tagged `transport: "lorawan"` with `loraRssi` and `loraSnr` from the gateway metadata, **omitting every null field**, because Losant rejects a null for a typed Number attribute and one null would void the whole report.
5. POST it to Losant, and log the exact outcome (resolved device, 200/401/404/422/502 with the rejection detail). The logging exists because TTS runs a circuit breaker: a webhook that fails repeatedly is auto-deactivated, after which the Netlify log goes quiet and the failure looks like "the webhook is not firing" when it is actually firing and failing (chapter 20, issue L8).

lorawan-switch-wifi.js brings a LoRaWAN node back to Wi-Fi. A LoRaWAN node is off Losant's MQTT, so a Losant command cannot reach it; the only path down is a class-A downlink delivered in the receive window after the node's next uplink. The function validates the session, finds the board's TTS device through the route table, and enqueues a one-byte `0x00` on fPort 2 via the Application Server's `down/replace` (replace, not push, so duplicate switch requests never stack). Delivery latency is bounded by the uplink interval, which is physics, not a bug.

register-gateway.js registers a customer's gateway on TTS under the GrowthPulse network. It validates the session, normalizes the 16-hex EUI, and creates the gateway entity through the

Identity Server with the operating cluster as its traffic server, treating "already registered" (409) as success so re-scans are idempotent. The boundary it cannot cross: the app can register the gateway entity, but the gateway hardware itself must already be configured to forward to the GrowthPulse network server (that is baked in before shipping; the app cannot reach inside a customer's gateway).

12.4 Environment variables

The complete set, with visibility:

Variable	Visible to	Purpose
VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY	browser bundle	Supabase project; RLS enforces all access
SUPABASE_SERVICE_ROLE_KEY	functions only	registry, routing, and ownership reads/writes that bypass RLS
LOSANT_APP_ID	functions only	Losant application id
LOSANT_API_TOKEN	functions only	read-only token (state and history)
LOSANT_COMMAND_TOKEN	functions only	write-capable token (commands, webhook state)
LOSANT_PROVISION_API_TOKEN	functions only	device-create and key-create rights
PROVISION_TOKEN	functions only	shared firmware token; must equal the value in the private firmware build
TTS_API_KEY	functions only	TTS key with end-device and gateway write rights
TTS_APP_ID	functions only	TTS application id (growthpulse)
TTS_CLUSTER	functions only	operating cluster, nam1.cloud.thethings.network
TTS_IS_HOST	functions only	Identity Server host, eu1.cloud.thethings.network
TTS_USER_ID	functions only	TTS account that owns registered gateways
TTS_FREQ_PLAN	functions only	US_902_928_FSB_2
LORAWAN_WEBHOOK_TOKEN	functions only	shared secret TTS sends as X-Webhook-Token

Operational fact that bit us more than once: **Netlify environment variables only take effect on a redeploy**. Set the variable, then trigger a deploy.

13. LoRaWAN engineering (The Things Stack)

13.1 Why LoRaWAN, and what it changes

Wi-Fi covers the home; LoRaWAN covers fields, far gardens, and greenhouses beyond Wi-Fi range: kilometers of reach and months of battery, at the price of a fundamentally different traffic model. LoRaWAN is not a streaming link. The Things Network fair-use policy allows about 30 seconds of uplink airtime per device per day (roughly one 9-byte frame every 15 minutes at the slowest data rate) and 10 downlinks per day, and a class-A node only listens in two short windows after each uplink. Every product decision around LoRaWAN, the binary payload, the queued switch command, the 15-minute staleness window in the app, follows from those constraints.

13.2 Topology: eu1 versus nam1

A critical, non-obvious fact about The Things Network: the **Identity Server**, which registers devices and gateways, is centralized on the **eu1** cluster, while a device's **Network, Join, and Application Servers** run on the operating cluster (**nam1** for North America). API calls therefore split across two hosts, and aiming a registration at nam1 fails with a misleading `route_not_found` (chapter 20, issue L3). Both hosts are configurable (`TTS_IS_HOST` , `TTS_CLUSTER`).

13.3 The four-call device API

Creating an end device through the API touches four endpoints, in order:

1. **Identity Server** (eu1): POST the registration, with the network, application, and join server addresses pointed at the operating cluster.
2. **Join Server** (nam1): PUT the root key (the AppKey lives here and only here) and `resets_join_nonces` , which is a Join Server field; the Network Server rejects it as a forbidden path (chapter 20, issue L14).
3. **Network Server** (nam1): PUT the MAC and PHY configuration: frequency plan `US_902_928_FSB_2` , LoRaWAN MAC 1.0.4, regional parameters RP002 1.0.4, `supports_join` .
4. **Application Server** (nam1): PUT the registration with an empty field mask.

Each server accepts only the field-mask paths it owns; sending one server a path that belongs to another is rejected as "forbidden path(s) in field mask" (chapter 20, issue L4). The split, the hosts, and the exact field masks are all encoded in `provision-lorawan.js` , which is the executable documentation of this API.

13.4 Gateways

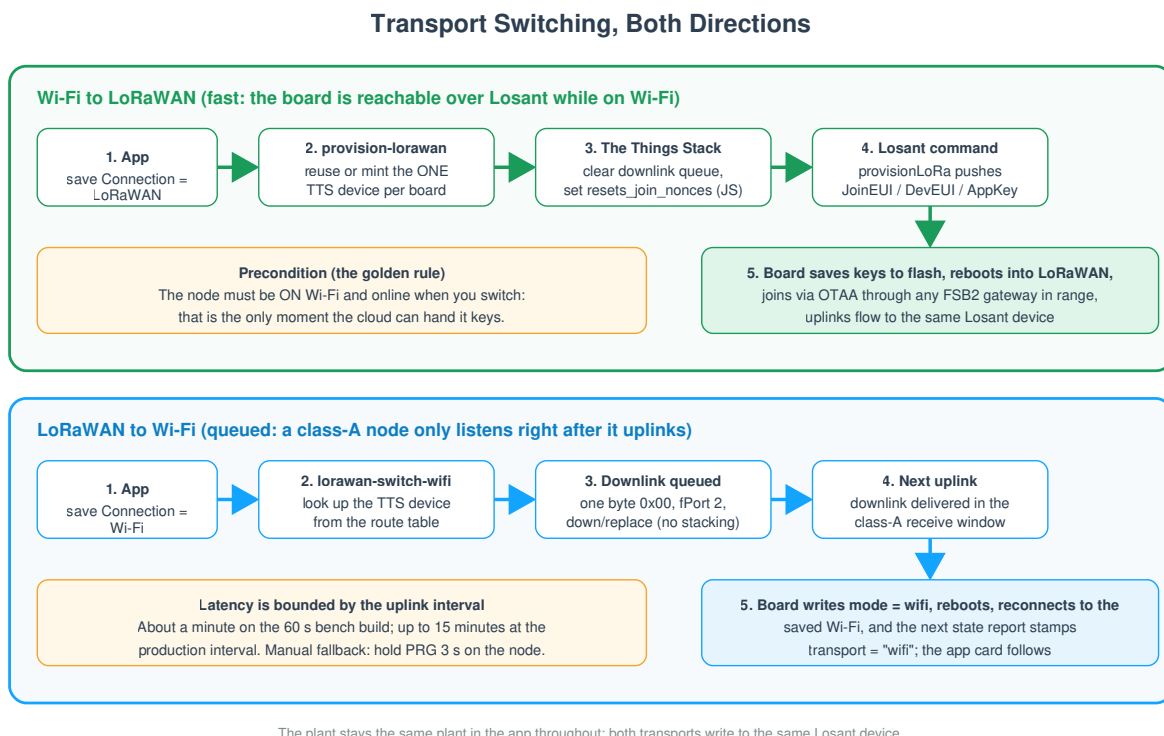
The development gateway is an Elecrow ThinkNode G1 (US915). It runs the Semtech UDP packet forwarder pointed at the operating cluster on port 1700. The single most important gateway fact: **the channel sub-band must be FSB2** (uplink channels 8 to 15), because that is what The Things Network uses in the US. The G1 shipped on FSB1, which meant the gateway literally could not hear the node: joins failed with -1116 while the gateway showed healthy and connected

(chapter 20, issue L1). Customer gateways must ship pre-configured for FSB2 and the GrowthPulse network server; the app's gateway registration handles the network side, and the customer never opens a console.

13.5 The webhook contract

One custom webhook per TTS application: JSON format, base URL `https://growthpulsecloud.com/.netlify/functions/lorawan-uplink`, the `uplink-message` event enabled, and an `X-Webhook-Token` header whose value equals the Netlify `LORAWAN_WEBHOOK_TOKEN`. The webhook's health banner in the TTS console is part of the system's diagnostics: TTS deactivates a webhook after repeated delivery failures, and a deactivated webhook with empty function logs is the signature of a downstream rejection, not a missing integration.

13.6 Transport switching, both directions



Wi-Fi to LoRaWAN is the easy direction, because the board is reachable: the app calls `provision-lorawan`, the backend ensures the one TTS identity exists, clears the downlink queue, and pushes the keys through a Losant command; the board saves them and reboots into LoRaWAN.

LoRaWAN to Wi-Fi is the hard direction, and three lessons shaped it:

- Delivery is a queued downlink that lands on the node's next uplink, so the switch takes up to one uplink interval. The app's edit sheet says so instead of pretending it is instant.
- The app gates the switch button on the **saved** transport, not the live card, because gating on the live reading once produced a deadlock: the card was stuck displaying Wi-Fi (issue L8), so the app refused to send the switch that would have fixed it (chapter 20, issue L7).

- A node stuck on LoRaWAN with no working downlink path would otherwise be unreachable, since provisioning requires Wi-Fi. The non-destructive 3-second PRG hold is the engineered escape hatch: it returns the node to Wi-Fi with credentials intact, from which every other operation is possible. The golden rule that falls out: **a switch to LoRaWAN must start from a node that is on Wi-Fi and online**, because that is the only time the cloud can hand it keys.
-

Part V. The Web Application

14. Application architecture

Stack: React 18 plus Vite, recharts for charts, lucide-react for icons, no router (a five-tab state machine), one hand-written CSS design system, Supabase JS for auth and data. Version 2.22.2.

14.1 Entry and the auth gate

`index.html` carries the PWA surface: viewport-fit for iPhone safe areas, theme color, manifest, icons, and the Apple metas that make Add to Home Screen produce a real full-screen app.

`App.jsx` contains `Gate`, the top-level decision: an "accounts service not connected" notice when env vars are missing, a loading state until the session is restored, the password-recovery screen when Supabase fires `PASSWORD_RECOVERY`, `Landing` and `Login` for signed-out visitors, or the app shell for a signed-in user.

The shell is mounted as `<AppProvider key={user.id}>`, and that `key` is load-bearing: changing user identity remounts the entire store, so no state can leak between accounts, or between demo and a real account.

Demo mode is a parallel fake session: the auth provider serves a synthetic demo user, so demo is just another user id to the rest of the app. That one mechanism gives complete data separation between demo and real data, with no flags sprinkled through the code. Demo runs on the Pro tier so prospects see everything.

14.2 The store

`store/AppContext.jsx` is one context providing everything; there is no Redux because one provider with useCallback actions is sufficient at this scale.

Persistence and isolation. `localStorage` keys follow `growthpulse:<userId>:<name>`. Combined with the provider remount, every account and the demo have fully isolated devices, settings, alarms, journals, gateways, and tier cache.

Cloud-authoritative devices. Real accounts load devices from the `devices` table (a `devicesReady` flag gates the UI while loading); every edit syncs back through `syncDevice`. Demo devices are the only ones persisted locally. A one-time migration lifts pre-ownership `localStorage` claims into the table.

The device model. Each device carries identity (id, name, location, geo, group, photo), configuration (transport, plant, irrigation including rain delay), claim linkage (`losantDeviceId`, `claimedAt`), and live state (reading, a rolling 60-entry history, `hasData`, `online`, `lastSeen`). Claimed devices initialize with a null reading and render "waiting for the first reading"; only real cloud data flips them live. This rule exists because an early build seeded claimed devices with simulated data, and fake-looking numbers on a real product destroy trust.

The live loop. One interval at the user's refresh rate (default 2 seconds) fetches `device-state` for every claimed device in parallel; errors keep the previous reading. Freshness flips `online` `false`

after 45 seconds of silence on Wi-Fi, 15 minutes on LoRaWAN (which reports on a minutes-scale cadence by design). A 15-second "connecting" grace window after load keeps a newly claimed device from flashing "Offline" before its first reading.

Weather, pinned to the plant. The selected device's stored geo drives an Open-Meteo forecast call. If the device has no geo, the store sets a `needsLocation` flag and stops: it never reads browser GPS and never invents a default city. The forecast describes where the plant lives, not where the owner is standing, which is the answer to the vacation problem, and location is requested only at the point of use by the one feature that needs it (rain delay).

14.3 Actions

The store exposes one action per intent: claiming (`claimDevice` : registry validation, optional geocode, insert with the 23505 duplicate-claim handling), editing (`updateDevice` with re-geocoding, `setDevicePlant` , `setIrrigation` , photo), lifecycle (`removeDevice` purges journals and device-scoped alarms and deletes the cloud row, releasing the claim; `factoryResetDevice` attaches the caller's session, POSTs the owner-verified command, then removes), LoRaWAN (`provisionLoRaWAN` , the Wi-Fi switch via `lorawan-switch-wifi` , gateway add with auto-registration), journals, alarms (including the one-tap plant-derived starter set), settings, and tier.

15. Domain logic

`store/helpers.js` is the domain brain:

- `METRICS` defines per-metric metadata: label, unit, icon, color, slider bounds, default good and warn bands. `rainChance` exists as a pseudo-metric so rain can participate in the alarm system without being a device sensor.
- `PLANTS` (in `plants.js`) is the catalog: ten categories (houseplants, tropical, succulents, herbs, vegetables, fruits, flowers, orchids, trees and shrubs, lawn), each with its own good and warn ranges for moisture, humidity, and temperature, and more than 140 species mapped into the categories with names, scientific names, and emoji. `rangesForDevice` layers the plant's ranges over the defaults, which is how "ideal" becomes species-relative everywhere at once.
- `statusOf` (inside good = good, inside warn = warn, else critical) colors the entire product through one function.
- `metricConnected` is the disconnected-sensor detector, built from measured failure signatures: soil temperature below -100, null values, humidity at or below zero (impossible from a working DHT22), soil raw under 300.
- `healthScore` averages good=100, warn=66, critical=28 over **connected** metrics only, so a missing probe neither drags nor props the score, and one critical metric visibly hurts (one critical among three healthy reads 82, not 95).
- `recommendations` orders the advice: missing-sensor fix-its first (naming the exact pin and component), then battery warnings (warn at 20 percent, critical at 10, suppressed while charging or on AC), then plant-care advice from the bands, then the all-good line only if nothing else fired.

- `activeAlerts` evaluates enabled rules per device (rain rules once against the shared forecast) and feeds the banner, badge, and toasts. Toasts are raised by key-diffing (`deviceId:ruleId`), so three simultaneous trips raise three named toasts rather than one anonymous notification.
- Unit handling converts at the display layer only; storage stays Fahrenheit so switching units never mutates data.

`store/tiers.js` defines the tiers as feature flags consumed across the app: Free (\$0: 3 devices, full plant catalog, live monitoring, alarms, history), Plus (\$4.99/mo: 10 devices, weather and rain intelligence), Pro (\$9.99/mo: effectively unlimited devices, automated irrigation with rain delay, LoRaWAN gateways). Real accounts default to Free with polished locked-state upgrade cards; the demo runs Pro.

16. Views and components

16.1 The five tabs

- **Live** is the dashboard: plant bar (tap to open the plant picker), three hero metrics, the moisture gauge with the health score, transport and signal and power badges, the weather card, the irrigation card with the rain-pause prompt, four sensor chips with trend arrows and tap-through detail charts, insights, and the growth journal. A claimed device that has not reported yet replaces the dashboard with a "waiting" state; an offline device shows "last reading X ago."
- **History** renders per-metric charts for hour, day, week, and month windows with the plant's ideal band shaded and max, average, and minimum per window.
- **Alarms** shows the live alert banner, the rule cards (metric, above or below, threshold slider, device scope, enable toggle), and the auto-set button that generates a plant-tuned starter set.
- **Devices** lists every plant with photo or emoji, pairing code, transport and signal, power, status, and moisture; groups become section headings. The edit sheet holds rename, location, group, photo with square crop, the transport switch with its honest latency note, and the danger zone (remove, factory reset) behind explicit confirmations. The gateways section lists account gateways and adds new ones by typed EUI or camera QR scan.
- **Settings** holds the plan card, units, theme (light, dark, auto), the LoRaWAN-aware refresh-rate pills with battery guidance, notification channels, and the account card with the report download and sign-out.

16.2 Honesty as UI policy

The honesty rules concentrate in the components: disconnected metrics render gray dashes with "not connected" and are not tappable; claimed-but-silent devices show waiting states, never numbers; the connection badge derives from the telemetry transport tag, never from the user's picker; battery percent and charging come from the device, and a unit with no battery shows AC. Each of those rules traces to a specific incident in the debugging journal where the UI briefly showed something that was not true.

16.3 The QR scanner, a camera lifecycle lesson

The gateway QR scanner (`QrScanner.jsx`) reads the gateway label and extracts the 16-hex EUI from the bundled label payload (`utils/eui.js` handles the URL-encoded multi-line format, colons and hyphens, and pretty-printing). Its bug is worth recording (chapter 20, issue A6): the scanner's effect depended on its `onResult` callback, and the Devices view passed a fresh inline function on every render, while the live polling loop re-renders that view every few seconds. The camera was therefore torn down and restarted continuously, which presented as "sometimes doesn't open, and when it opens it doesn't scan." The fix runs the effect once on mount and reads the latest callback through a ref, with `autoplay` and a metadata-load play retry for iOS Safari, where `play()` can be rejected outside the original tap gesture.

17. The report system

The report pipeline produces a branded, graph-rich PDF of any account's plants over any period, and it is the most layered subsystem in the app:

1. **ExportSheet** collects the options: period presets, custom dates, or "Everything" (back to each plant's `claimedAt`), which plants, which sensors. While building, it swaps to a spinner with phase labels and locks itself closed so an in-flight render cannot be orphaned or double-fired.
2. **History** comes from `device-history` per plant in parallel (mock series for demo plants), auto-bucketed server-side to about 240 points.
3. **buildReport** (`utils/accountExport.js`) assembles one branded HTML document: letterhead, account and period block, per-plant inline-SVG line charts (ideal-band shading, min/avg/max, segmented at nulls so offline gaps and dead probes show as honest gaps), a chronological activity timeline (claimed, first data, journal entries), and summary tables. All report CSS is scoped under `.gp-doc / .gpr-*` because the client-side renderer mounts the report inside the live app page, and unscoped class names once collided with the app's own (the centered-logo bug).
4. **Delivery, three paths sharing that one document:** Download PDF tries the server renderer first (`render-pdf` , true vector text) and falls back automatically to client-side `html2pdf` rasterization (dynamically imported, render scale clamped for mobile Safari's canvas limits) on any failure or in demo mode, so the download always succeeds. Print report opens the standalone HTML and auto-prints, which is also the sharpest save-as-PDF path. The older single-plant snapshot (`utils/report.js`) remains for quick shares.

18. The design system and PWA

`index.css` is the entire design language: a token set (background `#eef0f2`, card white, ink `#2c3e50`, brand green `#2ecc71`, status colors, 22px card radius, layered soft shadows, a 480px phone column), components (cards, badges, pills, chips, sheets with grab handles and slide-up animation, toasts, skeletons, the warnbox), and a dark theme as a token override on `data-theme` , with auto following the OS. At 860px the layout converts to a desktop grid: the bottom tab bar becomes a 240px sidebar. Two specific fixes are part of the record: form controls inherit the app font (browsers default them to system fonts with different metrics, which users perceive as "the typing looks shifted"), and badges are nowrap after a mid-word wrapping bug. Safe-area insets

pad the app bar, tab bar, and sheets so the installed PWA clears the iPhone notch and home indicator. The manifest and icons (rendered onto a white matte, because iOS renders transparency as black) complete Add to Home Screen.

Part VI. Integration, the Record, and the Road Ahead

19. End-to-end sequences

First boot and Wi-Fi setup. Power on; OLED "Starting up"; boot-time PRG check; self-test screen; watchdog armed; mode read from NVS (wifi). `connectWiFi()` finds no credentials and opens the branded hotspot; the customer joins it, the portal serves the config page (or 192.168.4.1), credentials save, association succeeds. `ensureProvisioned()` finds no identity, POSTs the pairing code, and stores the returned triplet. MQTT connects, the grace window opens, telemetry begins on the 3-second cadence, the pair-code page shows ON.

Claim. App: Connect a device, transport choice, name, optional location, code. Store: registry SELECT (RLS-permitted read), reject unknown codes; insert the `devices` row (UNIQUE `claim_code` enforces one owner; 23505 becomes "already claimed"); geocode if a place was given; select the new device. UI shows the waiting state until the first poll returns data.

Live tick. Interval fires; claimed devices fetch `device-state` in parallel; nulls and sentinels flow through untouched; history appends; trends, health, and insights recompute; `metricConnected` decides which slots render data versus "not connected."

Alarm trip. A reading crosses a rule. In-app: `activeAlerts` includes it, the badge and banner update, the key-diff raises a toast. Out-of-app: the Losant workflow's conditional passes and the email or SMS node fires, subject to the one-per-minute email rate limit. Two independent layers, by design.

Switch to LoRaWAN. Edit sheet, transport to LoRaWAN, save. The app POSTs `provision-lorawan` with the session; the backend reuses or mints the one TTS identity, clears the downlink queue, sets `resets_join_nonces`, pushes keys via `provisionLoRa`; the board (on Wi-Fi, past the grace window) stores keys and reboots; the SX1262 comes up, OTAA joins through any FSB2 gateway in range; uplinks flow gateway to TTS to webhook to Losant; the card flips to LoRaWAN with live RSSI and SNR within a poll.

Switch back to Wi-Fi. Edit sheet, transport to Wi-Fi, save. The app POSTs `lorawan-switch-wifi`; the backend queues the one-byte `0x00` via `down/replace`; the node's next uplink carries the downlink back; the firmware writes the mode and reboots; saved Wi-Fi reconnects; the next state report stamps `transport: "wifi"` and the card follows. Latency is bounded by the uplink interval.

Factory reset. Edit sheet, danger zone, confirmation with the data-loss disclaimer and the offline PRG fallback note. `factoryResetDevice` attaches the session, the function checks ownership through RLS and sends the allowlisted command, the unit wipes Wi-Fi on screen and reopens setup; the app removes the device, deletes the cloud row (releasing the claim), and purges journals and device-scoped alarms.

Report export. Settings, Download report, options, generate. Histories fetch in parallel; the document builds; the server renders a vector PDF or the client falls back; the file saves. The spinner with phase labels covers the wait.

20. The debugging journal

Every significant problem encountered during development, with symptom, root cause, and fix. This chapter is the part of the manual most worth reading slowly: the system's architecture is, to a large degree, the residue of these fixes.

Hardware and sensors

Issue H1. Soil temperature read -196.6 F. Root cause: the missing 4.7k pull-up on the DS18B20's 1-Wire data line (or an absent probe). -196.6 F is the Dallas library's -127 C disconnected sentinel. Fix: the resistor, plus end-to-end honesty so the app names the resistor when it sees the signature.

Issue H2. Soil moisture frozen at 100 percent, raw ~120, ignored water. Root cause: the capacitive probe was powered at 3.3V, and its NE555 oscillator does not run below about 4V; the power LED lit anyway. Fix: power the probe from 5V (output stays under ~3V, ADC-safe).

Issue H3. Soil ADC read a flat 0 on GPIO1. Root cause: GPIO1 is wired to the V3's battery-sense divider. Fix: move the probe to GPIO2; the ADC scanner sketch plus a 3V3/GND jumper test is how it was isolated. The freed conflict later became the battery feature.

Issue H4. Battery always read 0 percent; the unit claimed AC on battery. Root cause: the V3's battery-sense control pin (GPIO37) is inverted from the documented reference, so the divider was disabled exactly when the firmware meant to enable it. Fix (v3.7): correct polarity, 16-sample averaging.

Issue H5. No charging indicator exists in hardware. Root cause: the V3 exposes no charge-status pin. Fix (v3.7): infer charging from the smoothed voltage trend and top-of-charge level.

Issue H6. Battery percent leaped from 44 to 65 the instant a charger connected. Root cause: a charger inflates the pack's terminal voltage immediately; the naive curve read it as charge. Fix (v3.8): the displayed percent creeps at most 1 percent per reading.

Issue H7. No way to turn the unit off without unplugging. Fix (v3.9): double-tap PRG enters deep sleep; single press or RST wakes. Waking was also exempted from the boot-time Wi-Fi wipe, because the wake button is the reset button.

Issue H8. With USB power and no battery, a phantom climbing percent. Root cause: the charge chip pins the sense line near a full pack's voltage and there is no battery-detect pin; software cannot fully distinguish the cases. Fix (v3.10): report AC when the line is pinned at the top and not rising. The complete fix is the optional VBUS divider mod (v3.11).

Wi-Fi provisioning

Issue W1. Phones hung for a minute joining the setup hotspot. Root cause: with AP and STA both active, background STA retry scans and modem power-save pulled the radio off the AP

channel. Fix: `setWiFiAutoReconnect(false)` and `WiFi.setSleep(false)`; joins now complete in seconds.

Issue W2. The portal vanished mid-setup. Root cause: WiFiManager's 180-second default portal timeout restarted the AP under a slow phone. Fix: 600 seconds.

Issue W3. The captive portal never popped on some phones. Root cause: OS-level portal detection suppression (often with cellular data on). Fix: the portal answers at 192.168.4.1 directly, documented everywhere; turning off cellular data helps.

Issue W4. Uploads died mid-write ("the chip stopped responding"). Root cause: 921600 baud over marginal cables. Fix: upload at 115200; if needed, force download mode (hold PRG, tap RST).

Web app honesty and accounts

Issue A1. A junk pairing code created a ghost device. Root cause: no claim validation. Fix: registry lookup; unknown codes are rejected with a human message.

Issue A2. Claimed devices showed fake-looking values before ever reporting. Root cause: devices were seeded with simulated data. Fix: claimed devices start empty and honest, with a waiting state.

Issue A3. The subscription plan reset to Free on every login. Root cause: the tier was stored only in the browser. Fix (v2.16): persist it in Supabase user metadata.

Issue A4. LoRaWAN was faked in the UI. Root cause: interim code forced every node to display Wi-Fi and carried "in development" notes. Fix (v2.17): the app derives the true link and real signal from telemetry.

Issue A5. The onboarding screen rendered squeezed inside the desktop sidebar column. Root cause: it was mounted inside the shell grid and auto-placed into the 240px column. Fix: the full-page centered wrapper.

LoRaWAN radio and The Things Stack

Issue L1. Joins failed with -1116 while the gateway looked healthy. Root cause: sub-band mismatch. The ThinkNode G1 shipped on US915 FSB1; The Things Network uses FSB2. The gateway could not hear the node at all. Fix: set the gateway's channel plan to FSB2 and make it permanent.

Issue L2. The combined firmware "joined" but the first uplink failed with -1101. Root cause: `begin0TAA` only sets keys; `activate0TAA` performs the join, and the combined image never called it. Fix (v4.1): the activate retry loop and the 8-argument `sendReceive`, matching the proven standalone sketch.

Issue L3. Device creation failed with 404 route_not_found on nam1. Root cause: the Identity Server is centralized on eu1; nam1 has no registration route. Fix: registration to eu1, JS/NS/AS calls to nam1.

Issue L4. "Forbidden path(s) in field mask" while configuring the device. Root cause: each TTS server accepts only the field-mask paths it owns. Fix: trim the Network Server mask to MAC/PHY paths; give the Application Server an empty mask.

Issue L5. After switching to LoRaWAN, the app card stayed on Wi-Fi. Layered causes fixed in sequence: auto-provisioned devices had no TTS payload formatter, so the webhook bailed with 422 (fix, v2.20: the webhook decodes the 9 bytes itself); the route store could fail silently on a non-2xx Supabase response (fix, v2.21: check and surface it); and the true root cause was issue L8.

Issue L6. Switching back to Wi-Fi failed with no_device_session. Root cause: orphan TTS devices. Provisioning minted a new DevEUI on every switch, so a board accumulated a dozen dead identities and the downlink targeted one the node had never joined. Fix (v2.21): idempotent provisioning, one stable TTS device per board, keys stored and re-pushed, orphans cleared by the v2 migration. The most important architectural correction in the system.

Issue L7. The switch-to-Wi-Fi button never fired. Root cause: the app gated the switch on the live card state, which was stuck on Wi-Fi (L5), a chicken-and-egg deadlock. Fix: gate on the saved transport so the action always fires.

Issue L8. The card never left Wi-Fi, and TTS auto-deactivated the webhook. The true root of L5, found by reading the webhook's deactivation banner: Losant rejects an entire state report containing an attribute the device does not define. The board's device predated the `lorarssi / loraSnr / transport` attributes, so every LoRaWAN report was rejected with 502 while Wi-Fi reports passed; the repeated 502s tripped the TTS webhook circuit breaker, silencing the function logs. Fix (v2.22): `provision-device` syncs the full attribute schema on every provision, and the webhook logs every delivery outcome.

Issue L9. The node bounced endlessly between Wi-Fi and LoRaWAN, sometimes wiping its own Wi-Fi. Root cause: stale `provisionLoRa / factoryReset` commands redelivered the instant the board reconnected. Fix (v4.1): the 12-second command grace window.

Issue L10. A node stuck on LoRaWAN could not be recovered without losing Wi-Fi. Root cause: provisioning requires Wi-Fi, and the only manual escape (the PRG hold) was destructive. Fix (v4.1): the tiered button; 3 seconds is now a graceful, credential-preserving return to Wi-Fi.

Issue L11. A factory-reset board seemed to change identity, orphaning its route. Investigation showed the pairing code is chip-derived and stable, so the board reclaims the same Losant device; the apparent change was the orphan problem (L6) wearing a costume. Recorded takeaway: do not factory-reset during testing; each reset triggers a provision cycle that, before idempotency, churned routing.

Issue L12. A stale 0x00 downlink bounced the node to Wi-Fi right after every join. Root cause: a leftover switch downlink from earlier testing sat in the TTS queue and was delivered on the first uplink after each join. Fix (v4.2): provisioning clears the downlink queue; the firmware's overlapping 20-second guard was removed because it also suppressed legitimate switches.

Issue L13. Joins took about eight retries and two minutes after each reflash. Root cause: DevNonce. A reflash wipes NVS and restarts the counter; TTS silently drops joins until the counter

passes its remembered high-water mark. Fix: set `resets_join_nonces` (see L14) and drop the join retry from 30 to 10 seconds.

Issue L14. `resets_join_nonces` was never actually taking effect. Root cause: the backend was PUTting it to the **Network Server**, which rejects it as a forbidden field-mask path; it is a **Join Server** field, since join nonces are the Join Server's business. The error had been swallowed in the best-effort reuse path, and a new-device provision would have failed at that step outright. Fix (v2.22.2): the field moved to the `js/` endpoint in both create and reuse paths, with the device EUIs the Join Server requires. Verified directly: the Join Server PUT returns the device with `"resets_join_nonces": true`, and the board now joins on the first try after a reflash. The diagnostic path is worth remembering: the Network Server's "forbidden_paths" error names the rejected field, which is the API telling you which server owns it.

Issue L15. Switch-to-Wi-Fi took up to 15 minutes. Root cause: class-A physics, not a bug. A downlink rides the node's next uplink, and the interval was the production 15 minutes. Fix (v4.2): 60-second interval for bench and demo, with the documented instruction to raise it back toward 15 minutes for production fair-use.

Web app, late fixes

Issue A6. The gateway QR camera sometimes did not open, and when it did, it did not scan. Root cause: the scanner effect depended on its `onResult` callback, recreated on every render of a view that re-renders every few seconds from the polling loop; the camera was torn down and restarted constantly. Fix (v2.22.1): run the effect once on mount, read the callback through a ref, add `autoplay` plus a metadata-load play retry for iOS Safari.

Operational notes that bit us

- **Netlify environment variables only apply on redeploy.** Always trigger a deploy after changing one.
- **A stale `.git/index.lock`** recurred on the development machine and blocked commits; `rm -f .git/index.lock` clears it.
- **The TTS webhook health banner is a circuit breaker.** A webhook deactivated for repeated failures produces empty function logs, which reads as "not firing" when it is firing and failing. Check the banner first.
- **Losant email alerts are rate-limited to one per minute.** Space test readings accordingly.
- **GitHub occasionally rejects a push with a transient server-side `commit_refs` error.** The commit is safe locally; retry the push.

21. Decision log

The reasoning behind every significant decision, grouped by layer. Each is a sentence because the full context lives in the chapters above.

Identity and provisioning. The pairing code derives from the chip MAC: unique per unit at zero provisioning cost, readable off the screen, permanent, and stable across resets, which makes provisioning idempotent. The registry validates every claim so typos cannot create ghost devices, and it is the future factory database. Self-provisioning at first boot replaced flash-time credentials

so one identical image serves every board with no per-board secrets. The claim code is typed by the customer because during setup the phone is on the device's hotspot with no internet, so silent binding is impossible; the zero-typing token handoff is specced for later.

Hardware. The soil probe lives on GPIO2 because GPIO1 is the battery-sense net (measured flat 0). The probe is powered at 5V because its NE555 will not oscillate below about 4V, and its output stays ADC-safe. Sensor failure sentinels pass through the pipeline untouched because hiding them would mask hardware faults; the app converts them to fix-it advice instead.

Firmware. One file per image, reviewable top to bottom and trivially flashable. Mode changes write NVS and reboot rather than hot-swapping stacks, because only the chosen stack is ever initialized and RAM is the ceiling. The PRG button is tiered with a live on-screen hint, and its 3-second tier is deliberately non-destructive because it is the LoRaWAN escape hatch. Destructive cloud commands are ignored for 12 seconds after connect because broker redelivery is indistinguishable from a user action except by timing. The 60-second LoRaWAN uplink interval is a documented bench setting; 15 minutes is the production value, and the difference is the latency bound on every queued downlink.

Cloud. Both transports write to one Losant device so a node stays one plant across switches. TTS is a bridge, never a second source of truth. Provisioning is idempotent (one TTS device per board) because every downlink target and route becomes ambiguous otherwise. The webhook decodes payload bytes itself so nothing depends on per-device console configuration. The attribute schema re-syncs on every provision because Losant rejects whole reports over one undefined attribute. `resets_join_nonces` is set on the Join Server because that is the server that owns it, and it is what makes reflashed boards join instantly. All tokens live server-side; the `VITE_` prefix is the public/private line; RLS is the authorization engine; commands are allowlisted and owner-verified.

Web app. Claimed devices start empty, the connection badge derives from telemetry, disconnected sensors say so, and weather never reads GPS or invents a city: honesty is a feature, and each rule traces to an incident. Location is requested only at the point of use by the one feature that needs it. The store remounts on user change and namespaces storage per user, which gives account and demo isolation through one mechanism. Demo is a fake user id rather than a flag. The health score excludes disconnected sensors and weights 100/66/28 so a single critical metric visibly hurts. Reports ship as a branded document with real cloud history, vector-rendered server-side with a raster fallback so the download always succeeds. The transport switch is gated on saved state, not live state, because the live card can be wrong exactly when the switch matters most.

22. Known limitations and roadmap

1. **Raise the LoRaWAN uplink interval for production.** The one open hardening item from the bring-up: 60 seconds is a bench setting; The Things Network fair-use calls for about 15 minutes, with the corresponding switch-back latency. The constant is documented in the firmware.
2. **Zero-typing auto-bind.** The portal carries a one-time account token the unit presents on first connect, removing the typed claim code. The typed code is the shipping-grade interim.
3. **Web push notifications** for alarm trips, complementing the cloud email and SMS layer.

4. **Billing.** Tiers are fully functional feature gates; Stripe replaces the demo "buy" path.
5. **Family sharing.** Multiple accounts per garden, unblocked by cloud-authoritative ownership.
6. **Rain-delay actuation.** The flag, gating, and UX exist; wiring it to suppress real pump commands lands with the irrigation backend.
7. **Provisioning hardening.** Per-batch firmware tokens or device certificates, rate limiting on the provisioning endpoint, and TLS certificate pinning in the firmware.
8. **Group-level report selection,** a convenience once accounts run many nodes per zone.

Completed and absorbed into the chapters above: per-unit cloud provisioning, server-side ownership, LoRaWAN bring-up with automated provisioning and bidirectional switching, battery telemetry, and durable history.

23. Appendix

23.1 Links

Resource	URL
Live app	https://growthpulsecloud.com
App repository	https://github.com/BryanPuckettGH/growthpulse-app
Netlify	https://www.netlify.com
Supabase	https://supabase.com
Losant	https://www.losant.com
The Things Stack console (nam1)	https://nam1.cloud.thethings.network/console
Open-Meteo	https://open-meteo.com
Heltec V3 documentation	https://wiki.heltec.org
RadioLib	https://github.com/jgromes/RadioLib
ESP32 Arduino boards index	https://espressif.github.io/arduino-esp32/package_esp32_index.json
CP210x USB driver	https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers
Arduino IDE	https://www.arduino.cc/en/software

23.2 Project file inventory

Path	Contents
<code>firmware/GP_Combined/GP_Combined.ino</code>	Production image, v4.3: one image, Wi-Fi or LoRaWAN by NVS flag
<code>firmware/GP_Node/GP_Node.ino</code>	Wi-Fi-only self-provisioning image (reference)
<code>firmware/GP_LoRaWAN/GP_LoRaWAN.ino</code>	Standalone LoRaWAN sketch (the proven OTAA reference)
<code>firmware/GP_Provisioning/GP_Provisioning.ino</code>	Original captive-portal firmware (reference)
<code>firmware/diagnostics/</code>	ADC scanner and 1-Wire scanner bench sketches
<code>netlify/functions/</code>	The nine serverless functions (chapter 12)
<code>src/</code>	The React application (chapters 14 to 18)
<code>docs/growthpulse-supabase-schema.sql</code>	Pairing-code registry
<code>docs/growthpulse-devices-schema.sql</code>	Ownership table plus RLS (photo migration in <code>growthpulse-devices-photo.sql</code>)
<code>docs/growthpulse-lorawan-routes-schema-v2.sql</code>	LoRaWAN route table (current)
<code>docs/growthpulse-gateways-schema.sql</code>	Account-saved gateways
<code>docs/ manuals</code>	This manual, the Technical Reference Manual, the Operations Manual, and the User Manual, each as markdown plus PDF
<code>growthpulse-simulator/simulator.mjs</code>	Telemetry simulator
<code>CHANGELOG.md</code>	Complete version history, app and firmware
<code>README.md</code>	Developer onboarding

23.3 Identifier reference (non-secret)

Identifier	Value
Demo unit pairing code	4A7AC
Demo unit hotspot	GrowthPulse-04A7AC
Demo Losant device id	6a1fb486df527c8bf8d3324b
Serial baud	115200
Portal fallback address	192.168.4.1
TTS application id	growthpulse
Development gateway	ThinkNode G1, EUI E4:38:19:FF:FE:2A:58:80

Access keys, API tokens, the provisioning token, and the Supabase keys are intentionally absent from this document. They live in the private firmware build, the gitignored `.env`, and Netlify's environment store.

GrowthPulse Engineering. Internal and confidential. Keep credentials out of documents and repositories.